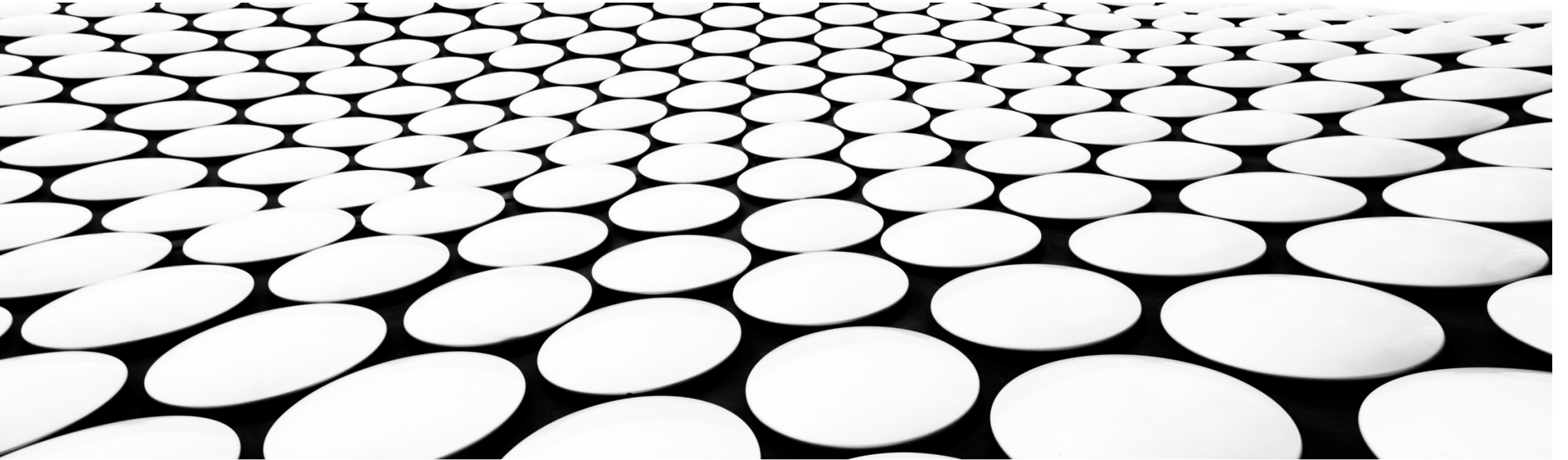

ARTIFICIAL INTELLIGENCE - STATE SPACE SEARCH



UNIFORM-COST SEARCH

- Instead of expanding the shallowest node, **uniform-cost search** expands the node n with the *lowest path cost* $g(n)$
- Expand least-cost unexpanded node-Cheapest-First Search
- Implementation:
 - *fringe* = queue ordered by pathcost
- Equivalent to breadth-first if path cost of all edges is same
- the goal test is applied to a node when it is *selected for expansion* rather than when it is first generated

Insert the root into the queue

While the queue is not empty

Dequeue the maximum priority element from the queue

(If priorities are same, alphabetically smaller path is chosen)

If the path is ending in the goal state, print the path and exit

Else

Insert all the children of the dequeued element, with the cumulative costs as priority

function UNIFORM-COST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

frontier $\leftarrow$ a priority queue ordered by PATH-COST, with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the lowest-cost node in *frontier* */

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

 add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

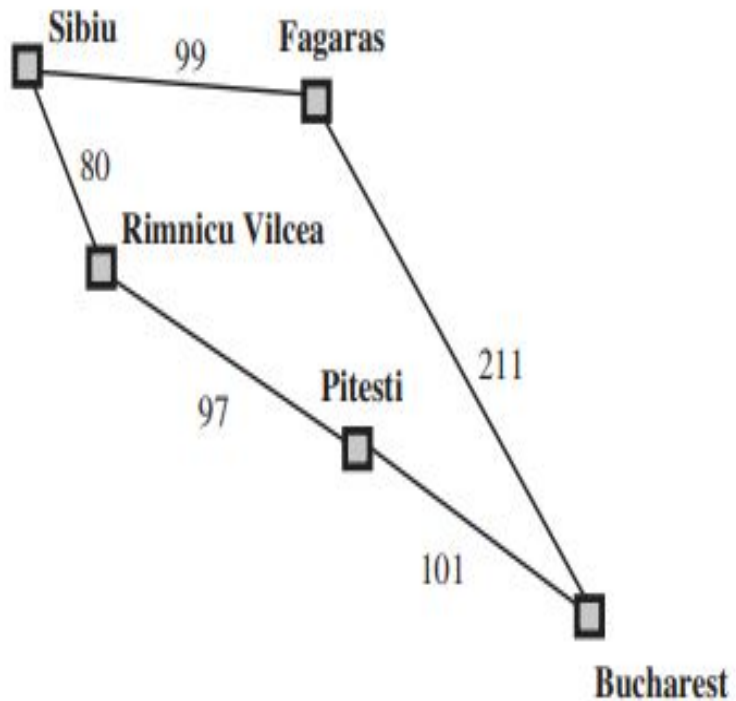
if *child*.STATE is not in *explored* or *frontier* **then**

frontier $\leftarrow$ INSERT(*child*, *frontier*)

else if *child*.STATE is in *frontier* with higher PATH-COST **then**

 replace that *frontier* node with *child*

UNIFORM COST SEARCH



- The successors of Sibiu are Rimnicu Vilcea and Fagaras, with costs 80 and 99, respectively.
- The least-cost node, Rimnicu Vilcea, is expanded next, adding Pitesti with cost $80 + 97 = 177$.
- The least-cost node is now Fagaras, so it is expanded, adding Bucharest with cost $99 + 211 = 310$.
- Now a goal node has been generated, but uniform-cost search keeps going, choosing Pitesti for expansion and adding a second path to Bucharest with cost $80 + 97 + 101 = 278$.

Part of the Romania state space, selected to illustrate uniform-cost search.

ALGORITHM-UNIFORM COST SEARCH

1. **Initialize:** set $OPEN=s$, $CLOSED=\{\}$ and $c(s)=0$
2. **Fail:** If $OPEN=\{\}$, terminate with failure
3. **Select:** Select a state with the minimum cost n , from $OPEN$ and save in $CLOSED$
4. **Terminate:** If $n \in G$, terminate with success
5. **Expand:** generate the successor of n

For each successor, m , :

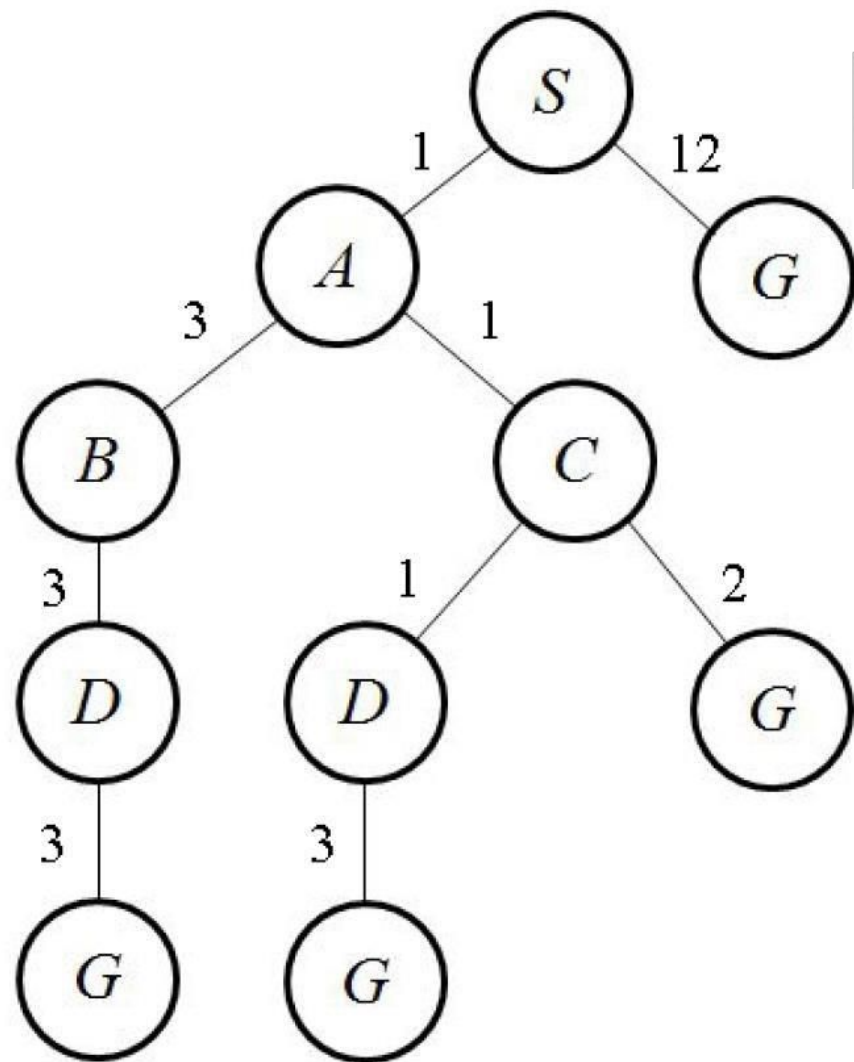
If $m \notin [OPEN \cup CLOSED]$

set $C(m) = C(n) + C(n, m)$ and insert m in $OPEN$

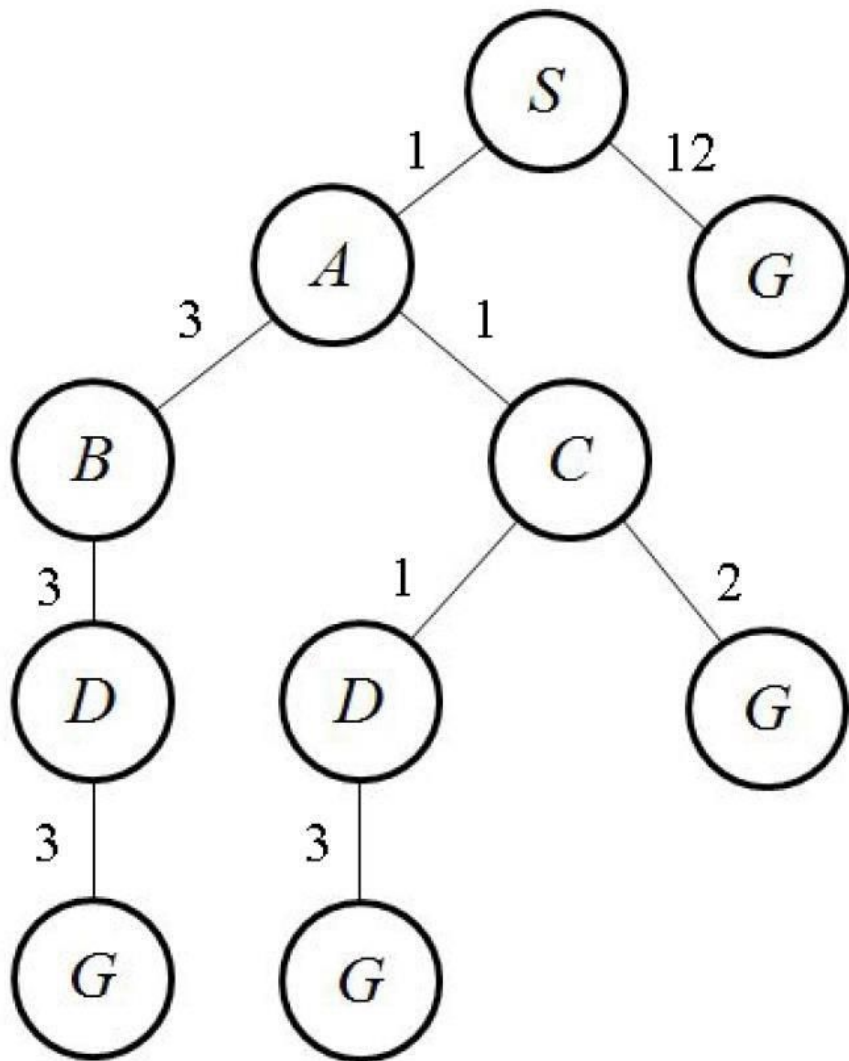
If $m \in [OPEN \cup CLOSED]$

Set $C(m) = \min\{ C(m), C(n) + C(n, m) \}$





Initialization: $\{[S, 0]\}$



Initialization: $\{ [S, 0] \}$

Iteration1: $\{ [S \rightarrow A, 1], [S \rightarrow G, 12] \}$

Iteration2: $\{ [S \rightarrow A \rightarrow C, 2], [S \rightarrow A \rightarrow B, 4], [S \rightarrow G, 12] \}$

Iteration3: $\{ [S \rightarrow A \rightarrow C \rightarrow D, 3], [S \rightarrow A \rightarrow B, 4], [S \rightarrow A \rightarrow C \rightarrow G, 4], [S \rightarrow G, 12] \}$

Iteration4: $\{ [S \rightarrow A \rightarrow B, 4], [S \rightarrow A \rightarrow C \rightarrow G, 4], [S \rightarrow A \rightarrow C \rightarrow D \rightarrow G, 6], [S \rightarrow G, 12] \}$

Iteration5: $\{ [S \rightarrow A \rightarrow C \rightarrow G, 4], [S \rightarrow A \rightarrow C \rightarrow D \rightarrow G, 6], [S \rightarrow A \rightarrow B \rightarrow D, 7], [S \rightarrow G, 12] \}$

Iteration6 gives the final output as $S \rightarrow A \rightarrow C \rightarrow G$.

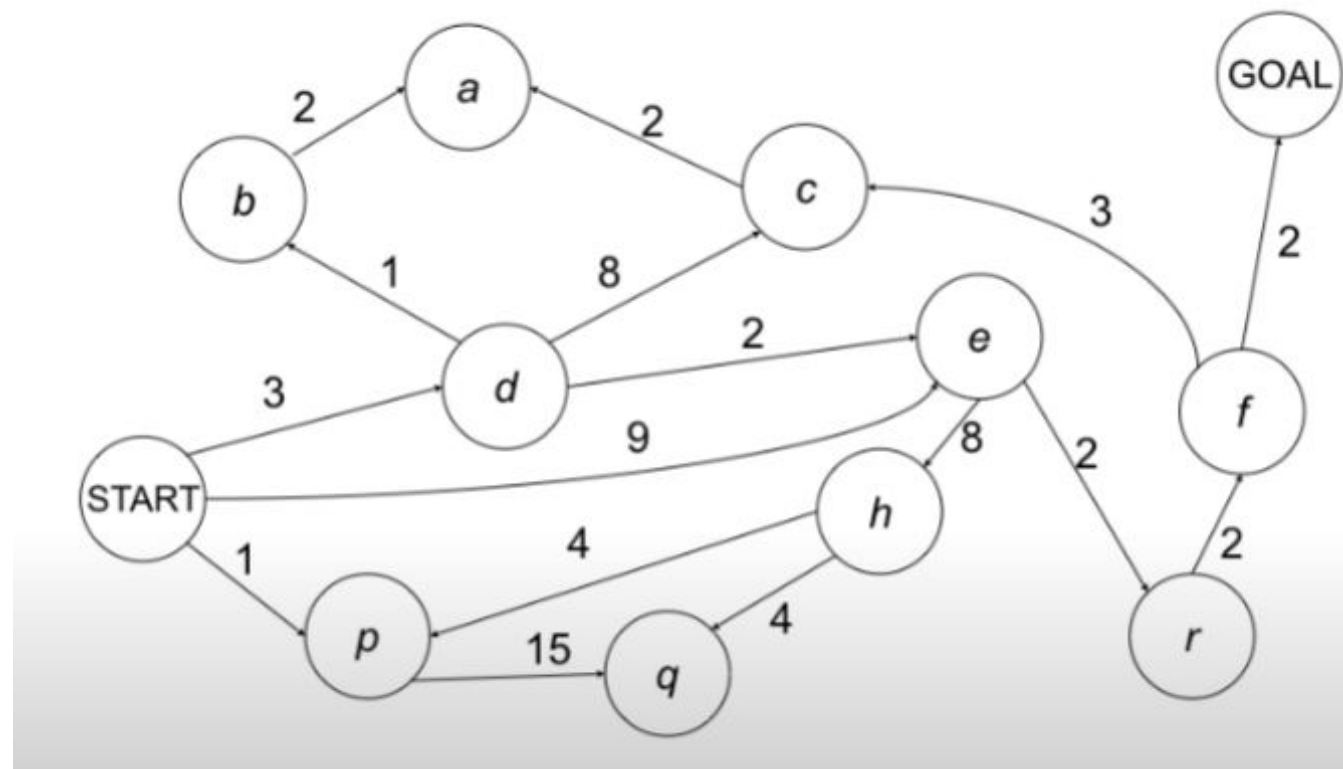
Nodes expanded: S,A,C,D,B,G

Solution path: S,A,C,G

UNIFORM-COST SEARCH

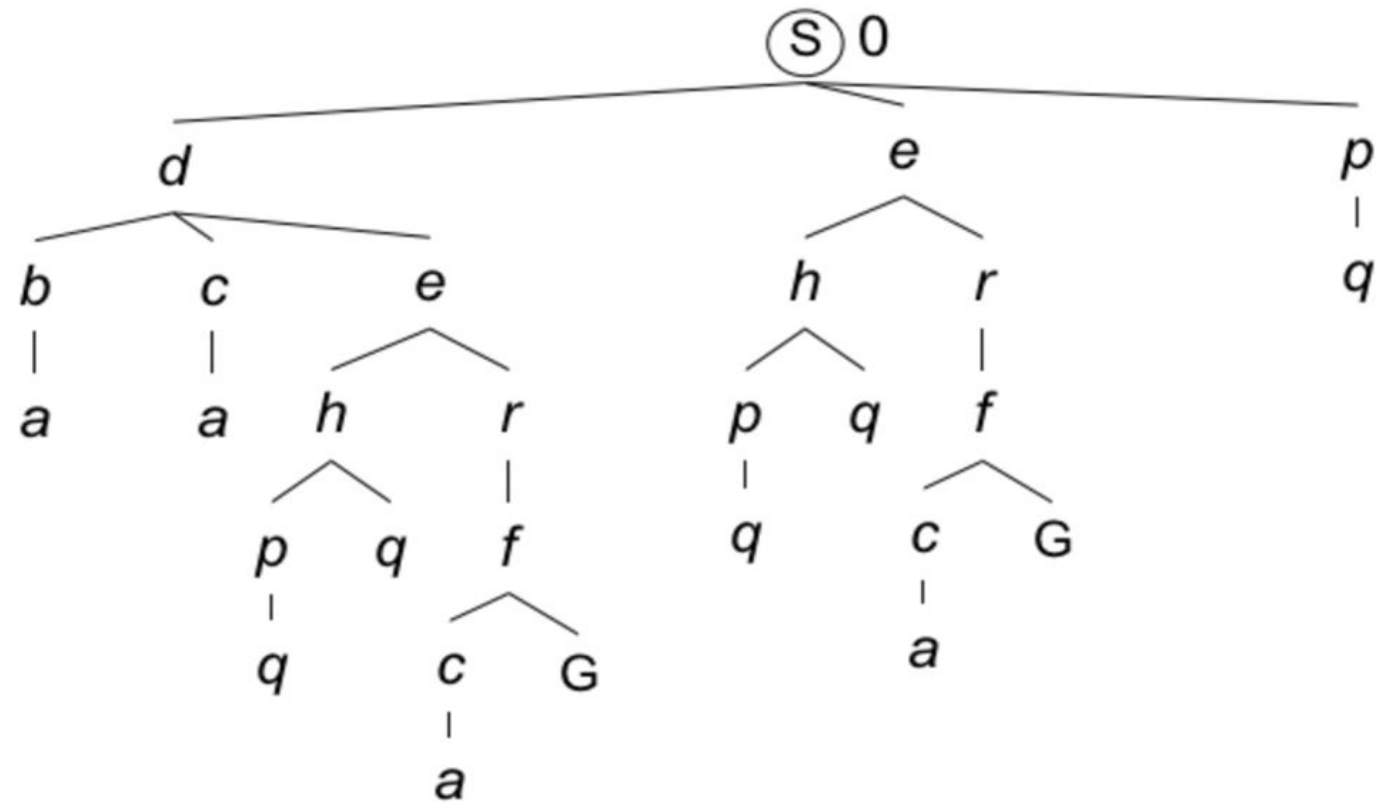
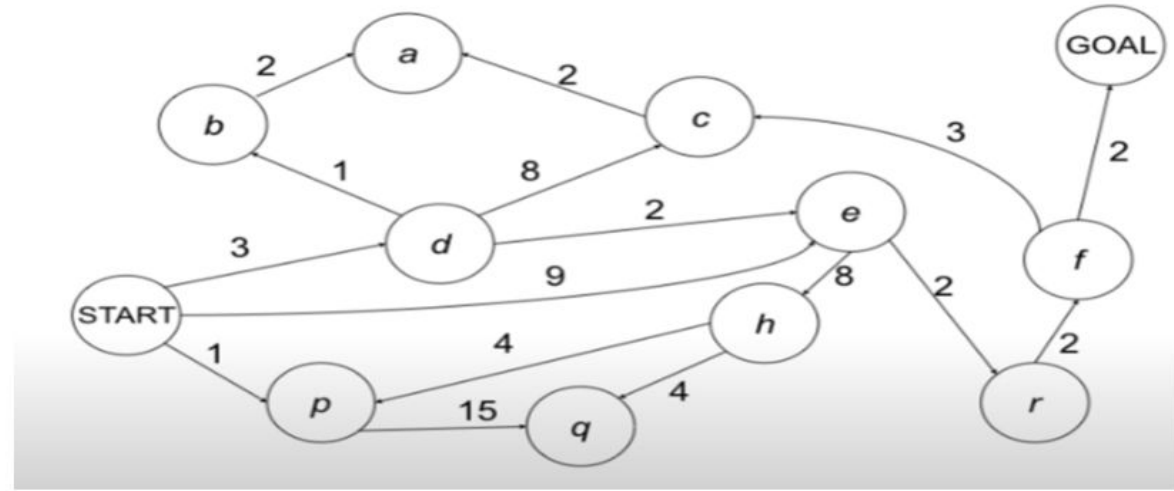
- *uniform-cost search expands nodes in order of their optimal path Cost*
 - whenever uniform-cost search selects a node n for expansion, the optimal path to that node has been found
 - step costs are nonnegative, paths never get shorter as nodes are added
- Uniform-cost search is guided by path costs rather than depths, so its complexity is not easily characterized in terms of b and d . Instead, let C^* be the cost of the optimal solution, and assume that every action costs at least ϵ .
- The algorithm's worst-case time and space complexity is $O(b^{1+\lceil C^*/\epsilon \rceil})$
- When all step costs are the same, uniform-cost search is similar to breadth-first search, except that the latter stops as soon as it generates a goal, whereas uniform-cost search examines all the nodes at the goal's depth to see if one has a lower cost

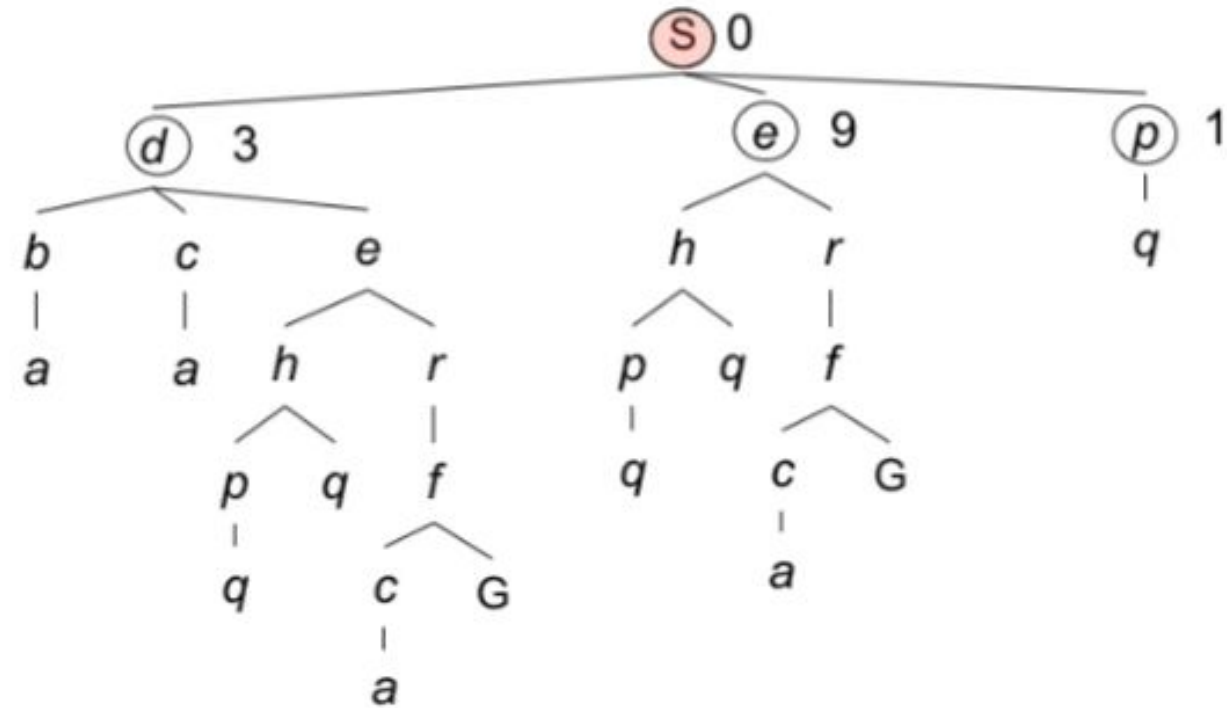
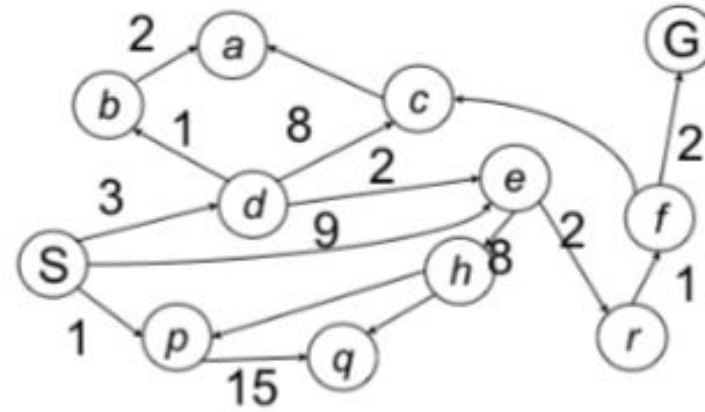
Algorithm	Complete?	Optimal?
BFS	✓ Yes	✓ Yes (if all step costs equal)
UCS	✓ Yes	✓ Yes (positive costs)
DFS	✗ No (in infinite spaces)	✗ No

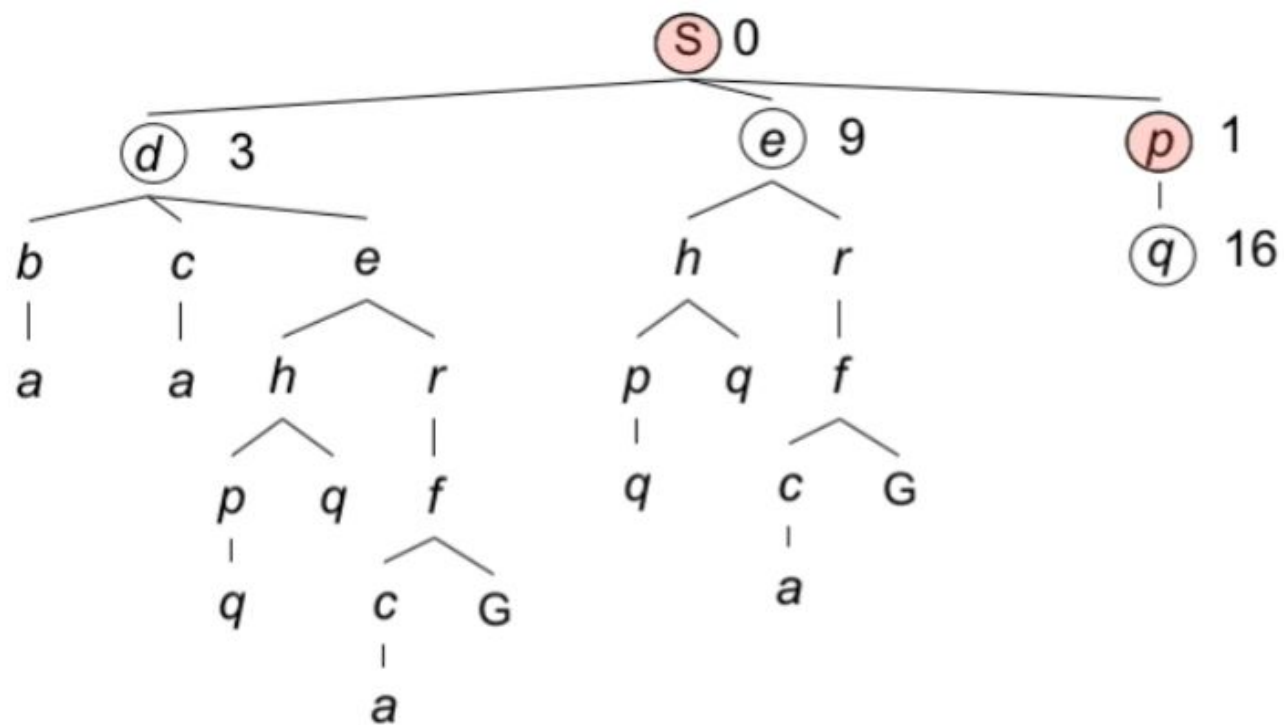
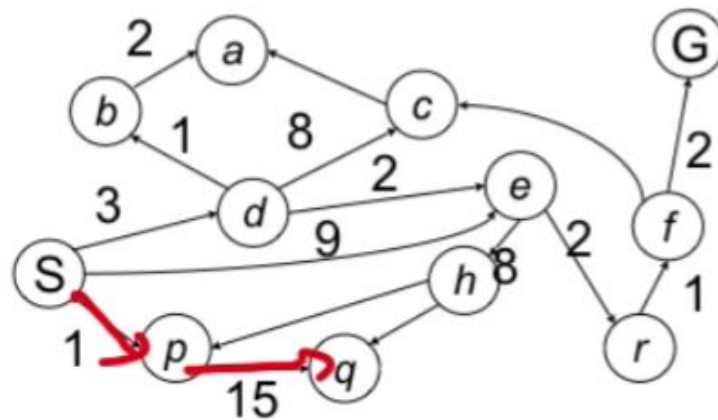


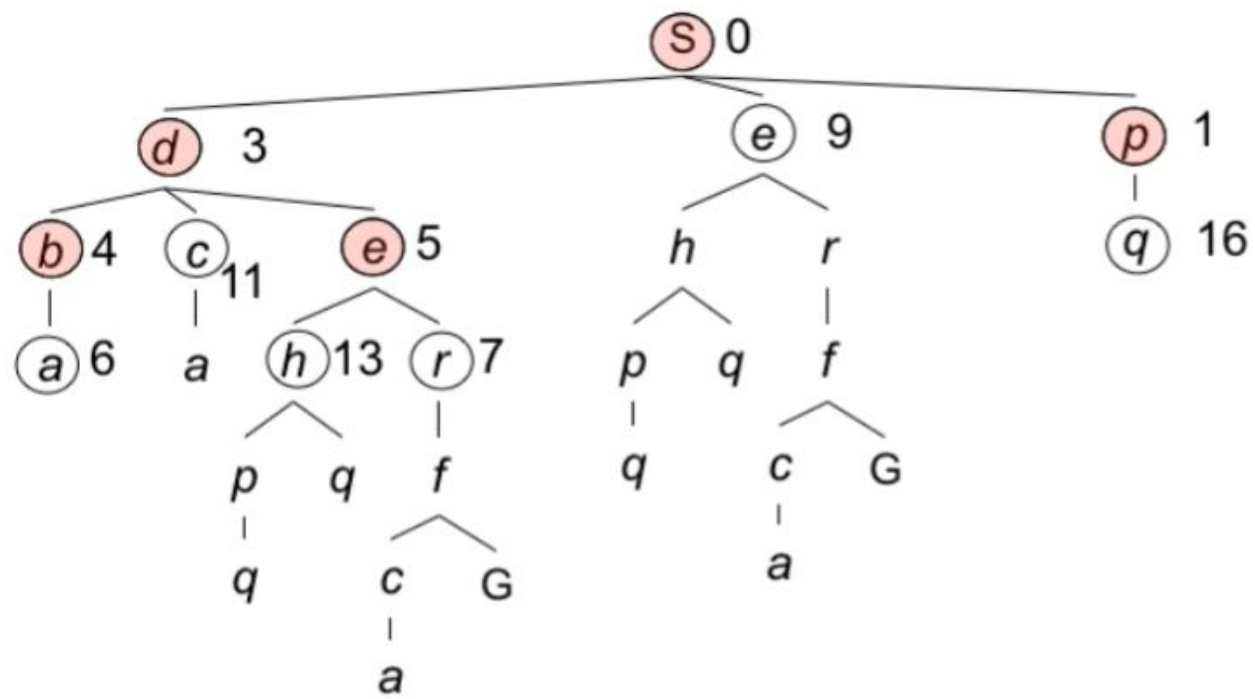
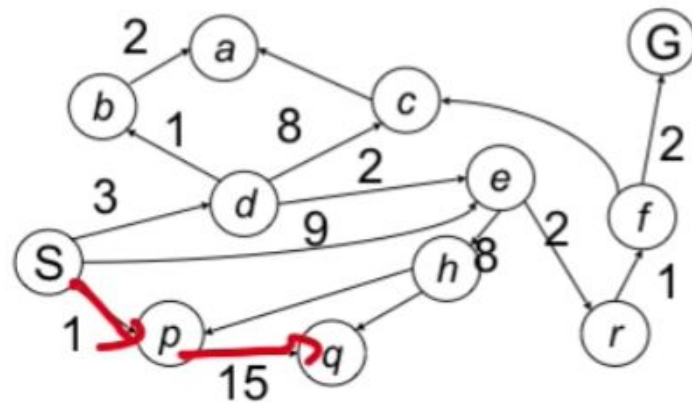
Strategy: expand a
cheapest node first:

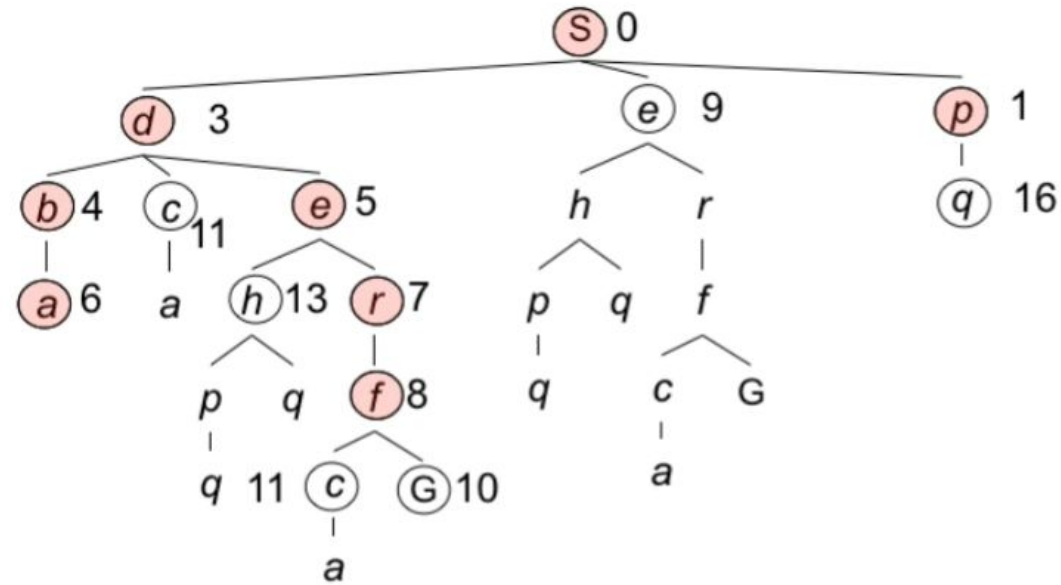
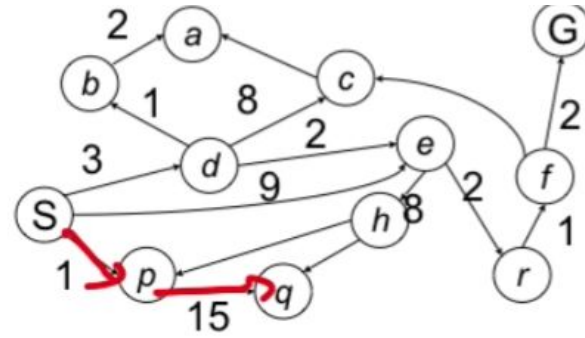
Fringe is a priority queue
(priority: cumulative cost)

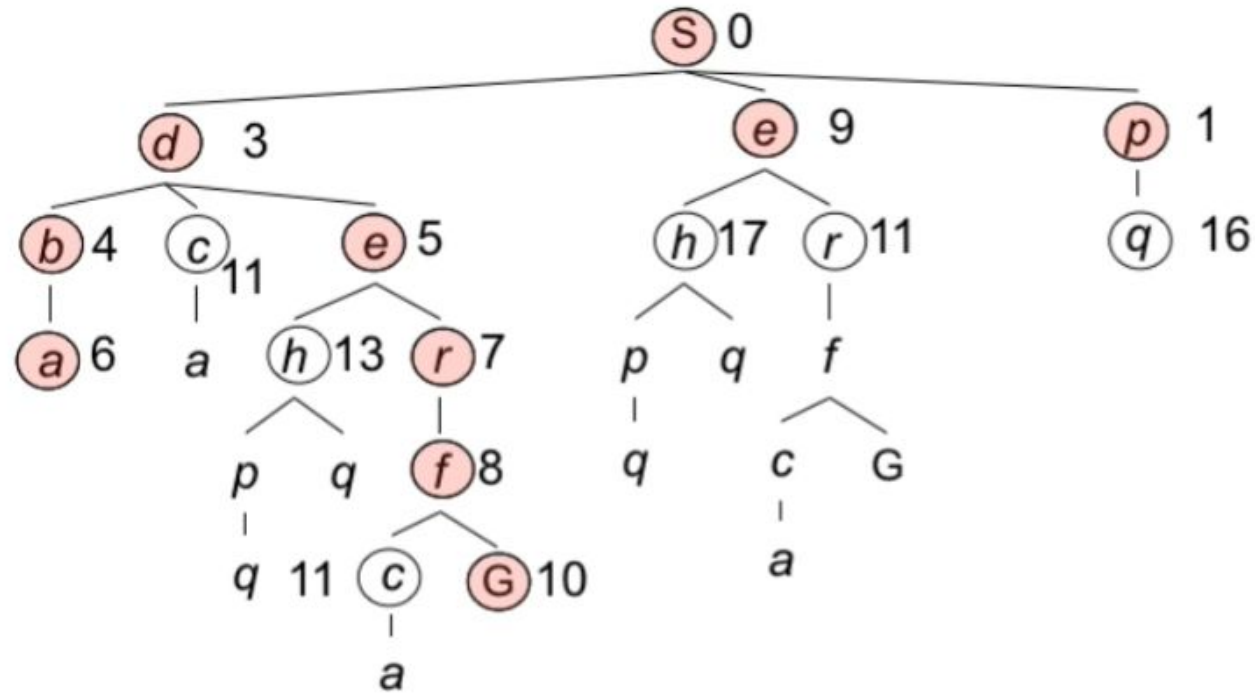
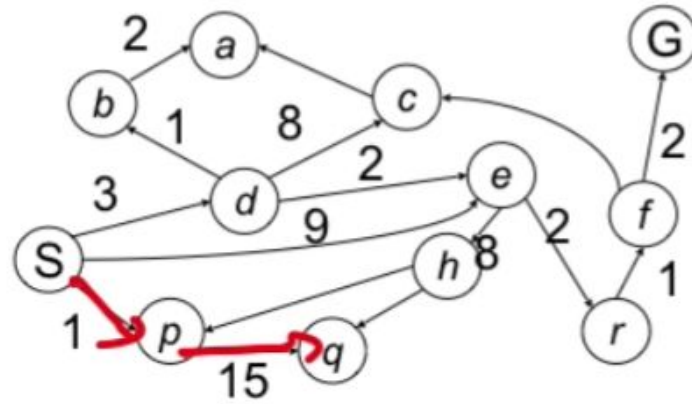






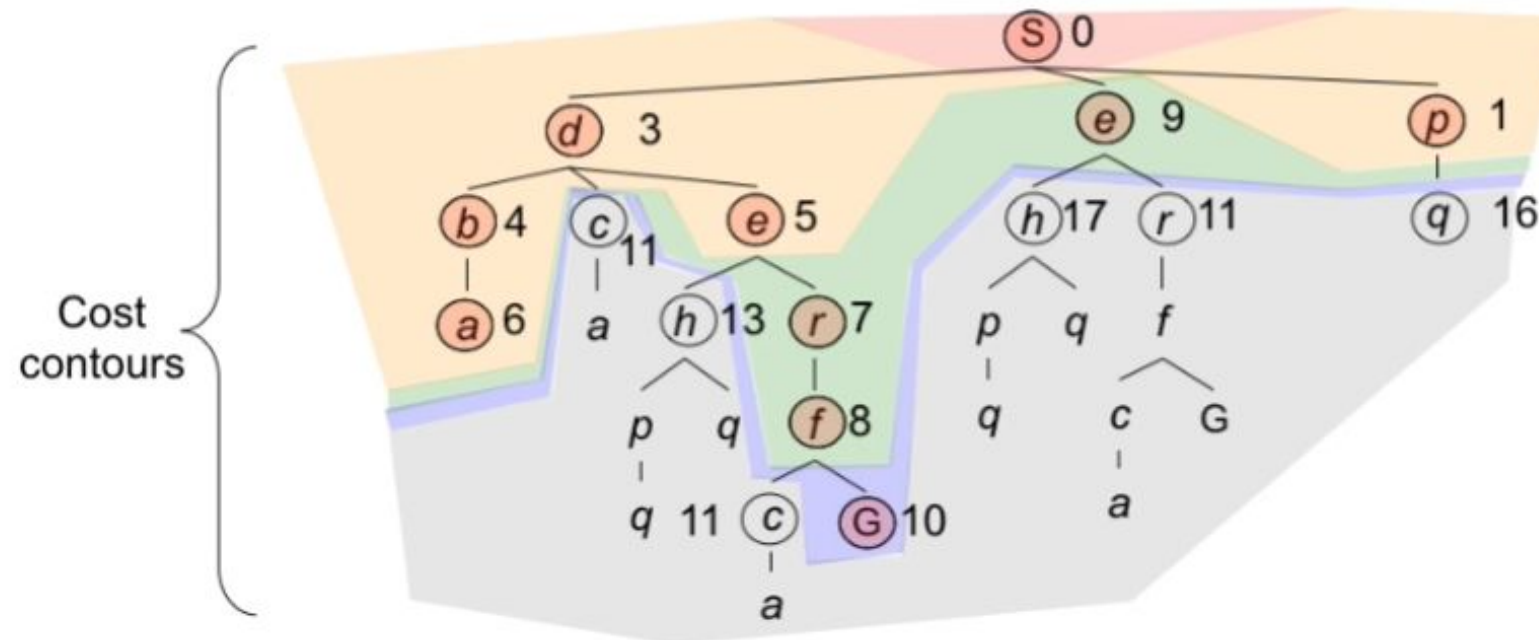
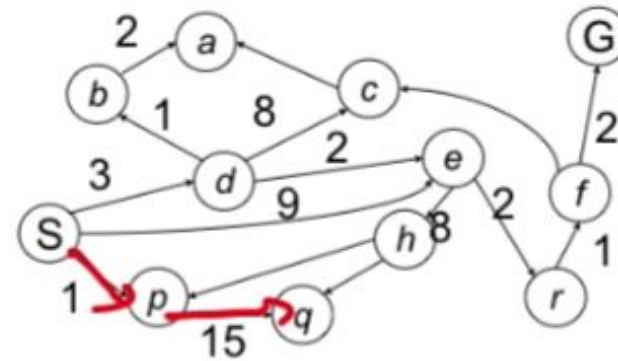






Strategy: expand a
cheapest node first:

Fringe is a priority queue
(priority: cumulative cost)



EXAMPLE: APPLY UCS(START STATE:V1, GOAL STATE: V6)

